/ MMB8052 - Practical 9

MMB8052 Practical 09 - Case Study IV: Downstream Gene List Analysis

Introduction

Lists of genes that change expression are of limited value taken on their own. Functional context is important for interpreting these findings, and can be added to lists of genes in a number of ways. We can look at expression patterns and group genes that vary in similar ways across our experiment. Clustering algorithms will find groups of genes that fulfil these criteria, and heatmaps give us a popular and usable way to visualize this kind of data. We can also exploit functional annotation that has been added to the gene databases of many organisms by both manual and automatic curation procedures. Examining gene lists for classes of annotation that are statistically overrepresented is a common way of ascribing functional importance to an otherwise hard to interpret list of genes.

It should be noted that this type of analysis is not restricted to the results of a transcriptomics experiment - overrepresentation analysis in particular can be applied in any analysis scenario where the end product is a list of genes identified by some selective process.

For the purposes of this practical, we will be picking up from where we left off in practical 8 - with the results of a `DESeq2` analysis of GSE116583 - an experiment looking at the gene expression changes associated with reperfusion in donor alveolar macrophages following murine lung transplant.

Clustering and Heatmaps

Clustering Algorithms

The goal of cluster analysis with RNA-Seq data is to group genes into clusters with similar profiles. The genes are assigned into clusters based on a dissimilarity measure (usually correlation or distance based). There are a number of algorithms for achieving this aim, they vary in their approach to the problem, but knowing which algorithm is best for the data being analysed is rarely simple.

Hierarchical Clustering

Hierarchical clustering is an agglomerative (“bottom-up”) approach where each gene starts in its own cluster (n genes, n clusters). The most similar genes according to a similarity metric (such as Euclidian distance) are merged (n genes, n-1 clusters), then the similarity metric is recalculated between the genes and this new cluster. This process is repeated iteratively until all genes are in a single cluster (1 cluster, of size n). Hierarchical clustering is implemented by the `hclust` function of the `stats` package in R and uses the output produced by the `dist` function.

Heatmaps

Heatmaps are a particularly useful post-analysis tool for visualising clusters in the genes that we know to be differentially expressed. The CRAN package `pheatmap` has a function `pheatmap()` that takes a numeric matrix as input. `pheatmap` applies hclust to cluster the rows and/or columns of the matrix, then displays a false colour image with dendrograms added to the left side and/or the top. These dendrograms are produced by applying hierarchical clustering to both the rows and the columns of the input matrix. In the case of expression data, this clustering will group together genes which share similar expression patterns across the samples, and samples which share similar profiles of gene expression.

Exercise 9.1

Estimated time: 15 minutes

- Using the results of the Allo24h contrast from practical 08, produce a heatmap for the expression of the 250 most differentially expressed genes (largest absolute log fold change, with a significant p-value)

Some hints to help:

- Use `dplyr::arrange()` to sort your filtered results by fold-change
- Remember this should be *absolute* fold change
- Use the `rlog` transformed data (which you used for PCA) for the values for the heatmap
    - `assay(rld)[filtered_results$ensembl_gene_id,]`

We also need to pay attention to the *scaling* of the values in the heatmap - by default `pheatmap` sets `scale="none"` - meaning the values plotted are exactly the same as the input values in the matrix. If, on the other hand, we use `scale="row"` or `scale="column"` then the numbers in the input matrix are Z-transformed in the appropriate axis (every observation in each row or column has the mean of the row/column subtracted from it, and the result is divided by the row/column standard deviation). This well-documented transformation can help if a small number of extreme observations dominate the heatmap, or if different sets of observations have very different distributions (often the case with different genes). A row-wise scaling means that we will be able to clearly see the differences in expression of every gene, but we can no longer compare genes to each other. Z-transformed data can also be thought of as 'diverging' (certainly more than untransformed data) as observations close to the mean will have a Z-score close to zero.

- Add `scale="row"` to your `pheatmap()` call

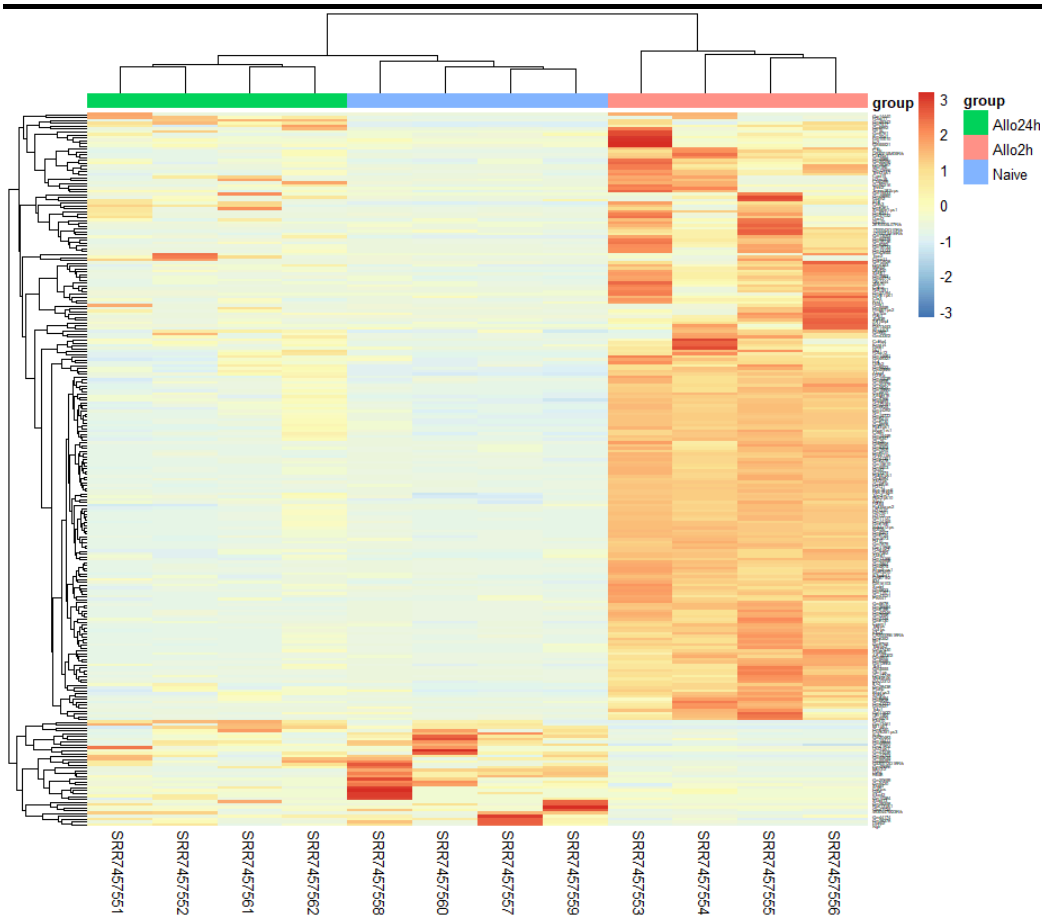


Figure 1: Example Heatmap from GSE116583

Enrichment Analysis

There are a number of reasons why we may want to consider the function of the genes in a gene list over and above the simple identification of the genes. Amongst these is the tendency for people to focus on those genes they recognize, and already suspect are interesting, at the expense of other unknown genes on the list that may be equally or more important.

Also, genes do not function in isolation - and the context of the genes on a list, both functional and physical, is equally important. The ways in which 'function' can be defined for genes are diverse, however it is generally accepted that we can use some common annotation that defines an arbitrary notion of function, and look for over-representation of that annotation among our list of differentially expressed genes.

The Hypergeometric Distribution

Suppose we have two lists of genes:

1. Genes differentially expressed in our experiment (N=200)
2. Genes found in organism of interest (N=20,000)

If we consider genes involved in the function "transcription activation" (for instance), we find the following:

1. 20 genes in list 1 have this annotation
2. 650 genes in list 2 have this annotation

What is the probability that we picked 20 genes with this annotation in list 1, given that list 2 contains 650 genes with this annotation?

The answer to this question is given by the hypergeometric distribution, a discrete probability distribution which describes the number of successes in a series of draws from a finite population without replacement (compare this with the binomial distribution, which describes the same but with replacement so that the probability of success at each draw is the same).

The hypergeometric distribution is often described in terms of an ‘urn model’ describing the number of white balls drawn without replacement from an urn which contains white and black balls. The R help page for the hypergeometric distribution uses this conceptualisation (see `?dhyper`). We can use a test against the hypergeometric distribution to determine whether the above annotation is statistically over-represented:

```
# Urn contains m white balls and n black balls
# k balls are drawn without replacement , x of these
> dhyper(x = 20, m = 650, n = 19350, k = 200)
[1] 6.38656e-06
```

The Gene Ontology (GO)

The Gene Ontology is a controlled vocabulary (CV) of terms for describing gene product characteristics and gene product annotation data. Actually, GO provides three controlled vocabularies, one each for *biological process*, *molecular function* and *cellular component* annotation. The terms from these CVs are attached to gene products by several mechanisms, including expert annotation and inference from sequence similarity. If you work with a standard model organism, the chances are that the state of GO annotation is pretty good, so a lot of value can be had from over-representation analysis using the hyper-geometric test and the Gene Ontology.

The Kyoto Encyclopaedia of Genes and Genomes (KEGG)

In many ways, the annotation available from the Kyoto Encyclopaedia of Genes and Genomes (KEGG) is similar to that in GO – functional annotation attached to genes. The KEGG database attempts to organize genes into biochemical pathways, rather than the somewhat broader functional classifications found in GO. The data can be used in exactly the same way in conjunction with the hypergeometric test.

The Molecular Signatures Database (MSigDB)

MSigDB is developed and distributed by the Broad Institute for use with their tool for analysing functional enrichment (called Gene Set Enrichment Analysis, or GSEA - more on this later). MSigDB is extremely extensive, cataloging 10s of 1000s of annotated gene sets, and it includes sets matching the databases already described above (particularly GO). In addition to these though, MSigDB contains gene sets which describe cell signatures and oncogenic processes, among others. A particularly useful group of gene sets is the so-called *hallmark* gene sets, which are designed to describe well-defined biological states or processes.

clusterProfiler

The enrichment analysis described above is a generic strategy which is relatively easy to apply, and could be achieved using the base R hypergeometric functions (like `dhyper()`, as shown above). Why then, should we use a specialist package for this type of analysis? There are two main reasons for this, 1) the convenience of access to appropriate annotation; 2) good quality visualisations. The `clusterProfiler` Bioconductor package provides both of these things.

Gene Ontology Enrichment Analysis

`clusterProfiler` has a function for carrying out Gene Ontology enrichment analysis, `enrichGO()`. The inputs for this function are a list of differentially expressed genes, and an annotation database which enables the mapping of genes to GO terms. For our mouse experiment, we can use the Bioconductor annotation package `org.Mm.eg.db`:

```
> install(c('clusterProfiler', 'org.Mm.eg.db'))
> library(clusterProfiler)
> library(org.Mm.eg.db)
> ego = enrichGO(gene=ens_gene_list,
                 OrgDb = org.Mm.eg.db,
                 keyType = 'ENSEMBL',
                 ont = "CC",
                 pAdjustMethod = "BH",
                 pvalueCutoff = 0.01,
                 qvalueCutoff = 0.05))
> head(ego)
```

The code in the block above calculates enrichment in the Cellular Component branch of the Gene Ontology for the genes in `ens_gene_list` (compared to all the genes in the mouse genome). We should make this more specific to our analysis by including a `universe` argument which lists just the genes we analysed for differential expression. To use the example of the 24 hour time point vs naive in the analysis from practical 08:

```
> results_table = results(dds, contrast= c('Group',
> results_tibble = as_tibble(results_table, rownames
> filtered_results = filter(results_tibble, complete
> significant_results = filter(filtered_results, abs
> significant_genes = pull(significant_results, ense
> universe_genes = pull(filtered_results, ensembl_ge
> ego = enrichGO(gene=significant_genes,
                 universe=universe_genes,
                 OrgDb = org.Mm.eg.db,
                 keyType = 'ENSEMBL',
                 ont = "CC",
                 pAdjustMethod = "BH",
                 pvalueCutoff = 0.01,
                 qvalueCutoff = 0.05)
```

Exercise 9.2

Estimated time: 20 mins

- Repeat the `enrichGO()` function for different branches of the Gene Ontology (BP and MF)
- Repeat for the Allo2h comparison and compare the top GO terms for each

clusterProfiler Visualisations

The `clusterProfiler` package comes with a range of visualisations built for viewing the results of the kind of enrichment analysis carried out in exercise 9.2. The table below lists some of the functions and what they show.

Function Name	What does it show?
`goplot()`	Shows the section of the Gene Ontology *Directed Acyclic Graph* (DAG) which contains the enriched terms (highlighed with a colour representing the p-value)
`barplot()`	Bar height shows the number of genes with that classifiction. Colour represents p-value.

Function Name	What does it show?
`dotplot()`	Similar to the `barplot()` in that dots are coloured by p-value. This time the gene number is encoded by the size of the dot, and the X position gives the ratio of genes in the whole geneset compared to the given list.
`cnetplot()`	Links enriched categories to the genes found in it. Can show genes shared between multiple categories.
`heatplot()`	Conceptually similar to a heatmap, but used to show group membership. Can show fold change as colour if provided with a `foldChange` argument.

Exercise 9.3

Estimated time: 20 mins

- Test out the functions listed in the table with the results generated in exercise 9.2
- Can you get the fold changes displayed on the `cnetplot()` and `heatplot()`?

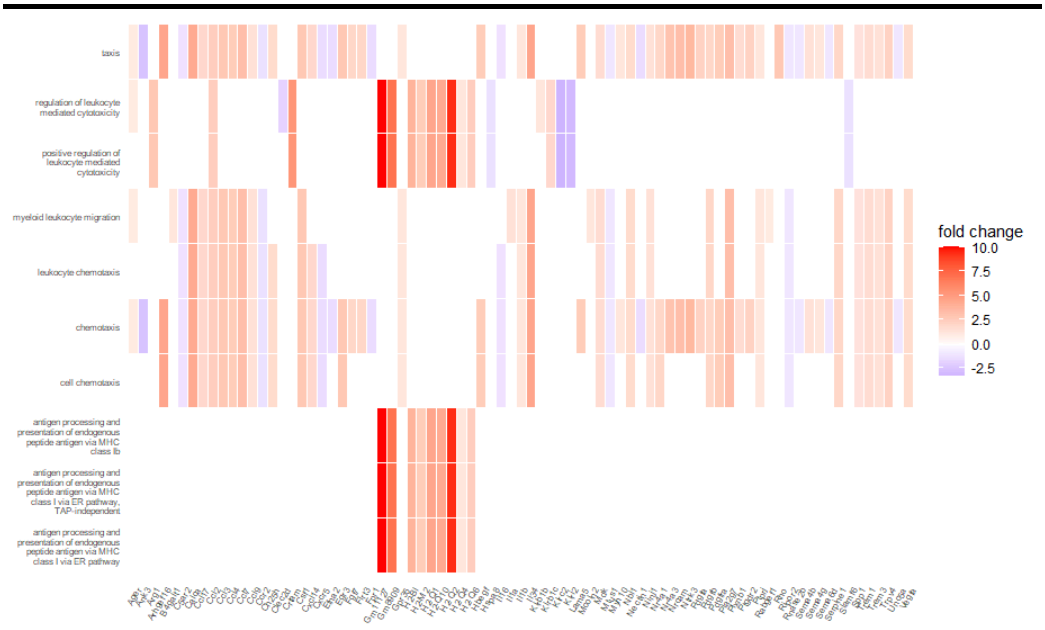


Figure 2: Example `heatplot()` from GSE116583

Gene Set Enrichment Analysis

Using fold change and p-value thresholds to determine genes of interest will find genes where the difference in expression between conditions is large. However, relevant differences may be present in the form of small but consistent changes in predefined sets of related genes. Gene Set Enrichment Analysis (GSEA) (Subramanian et al. 2005) is a computational method that can determine whether genes in predefined sets change in a small but coordinated way.

Briefly, given a predefined set of genes S, and a list of genes L ranked according to some metric, GSEA will determine if the genes in S are randomly distributed throughout the ranked list L, or if they are over-represented at the top or bottom. GSEA does this by walking down the list L and increasing a running-sum statistic when a gene in S is encountered, and decreasing it when it is not. The significance of the resulting enrichment score is calculated via a permutation test.

The `clusterProfiler` package contains an implementation of the GSEA method. We will use it with the MSigDB "Hallmark" gene sets here.

```
> gmt = gson::read.gmt("https://raw.githubuserconten
> for_gsea = pull(annot_results, log2FoldChange)
> names(for_gsea) = pull(annot_results, external_gen
> for_gsea = sort(for_gsea, decreasing = TRUE)
> gsea = GSEA(geneList = for_gsea,
              exponent = 1,
              nPerm = 10000,
              minGSSize = 5,
              maxGSSize = 500,
              pvalueCutoff = 0.05,
              pAdjustMethod = "BH",
              TERM2GENE = gmt,
              by = "fgsea")
```

Exercise 9.4

Estimated time: 20 mins

- Carry out GSEA as above, with the 2h and 24h gene lists
- Explore the results
- Use `ridgeplot()` and `gseaplot()` to visualise the results

NOTE: `gseaplot()` needs a geneset to visualise, to plot the top-ranked geneset, try the following:

```
gseaplot(gsea, geneSetID = 1, title = gsea$Descripti
```

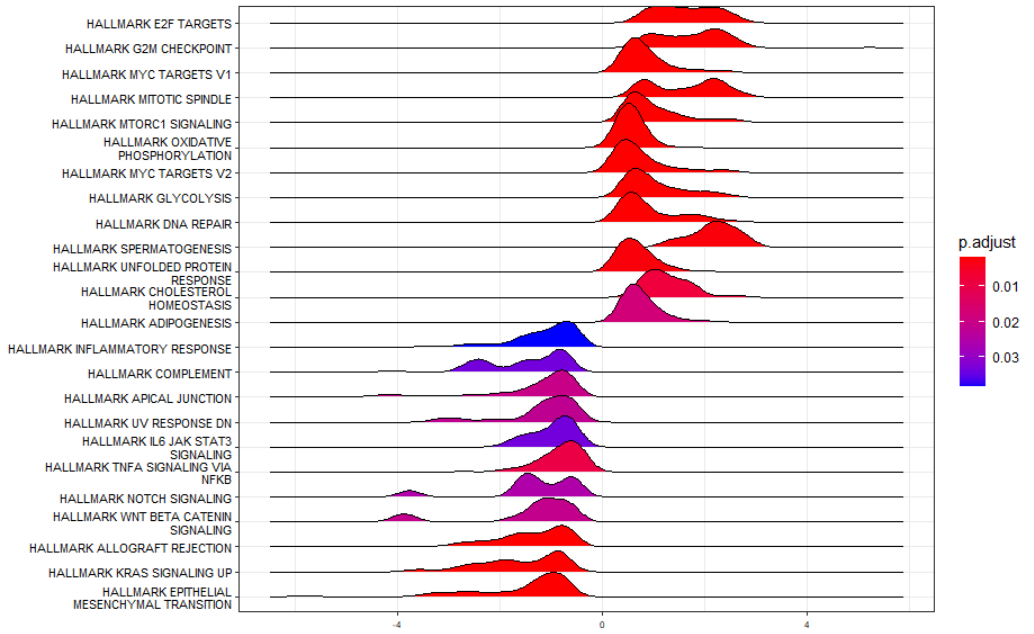


Figure 3: Example `ridgeplot()` from GSE116583

Consider the following:

- What's the relationship between the enriched gene sets at the two timepoints?
- Are the GO terms and the Hallmark genesets related in any obvious way?

Resources on the web for downstream analysis

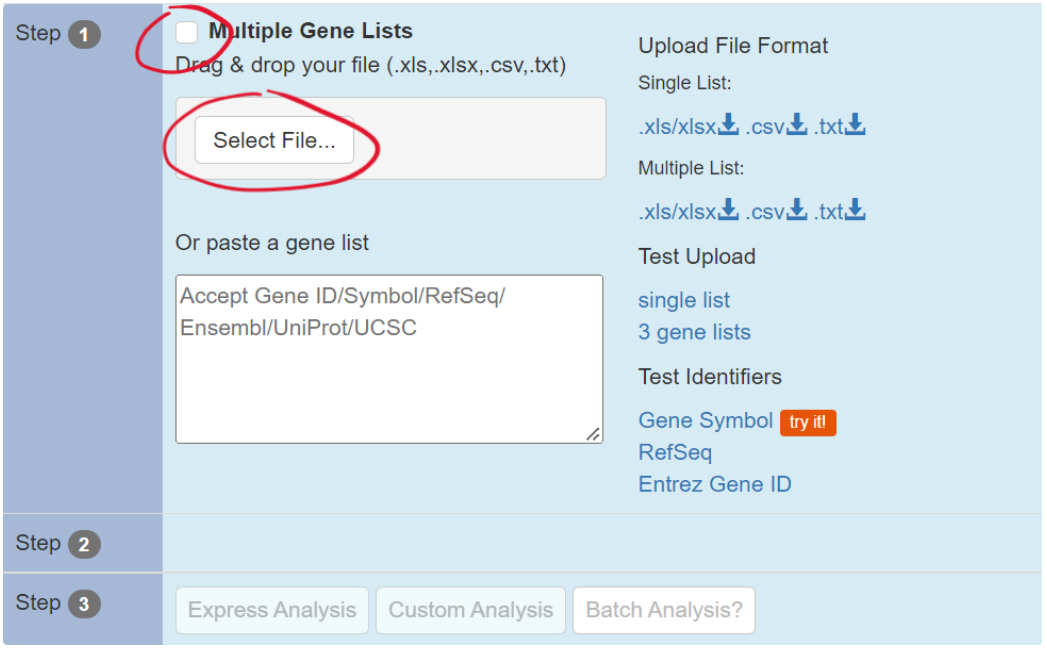
Just because we have spent the semester learning about Linux and R does not mean we are bound to use only these tools. There are lots of fairly user-friendly tools out there on the web that enable lots of interesting analysis. The main restrictions of web tools tend to be that we cannot use them programmatically, so they do not fit in to a coded pipeline of analysis, and that they can go out of date fairly quickly if they are not maintained.

Metascape

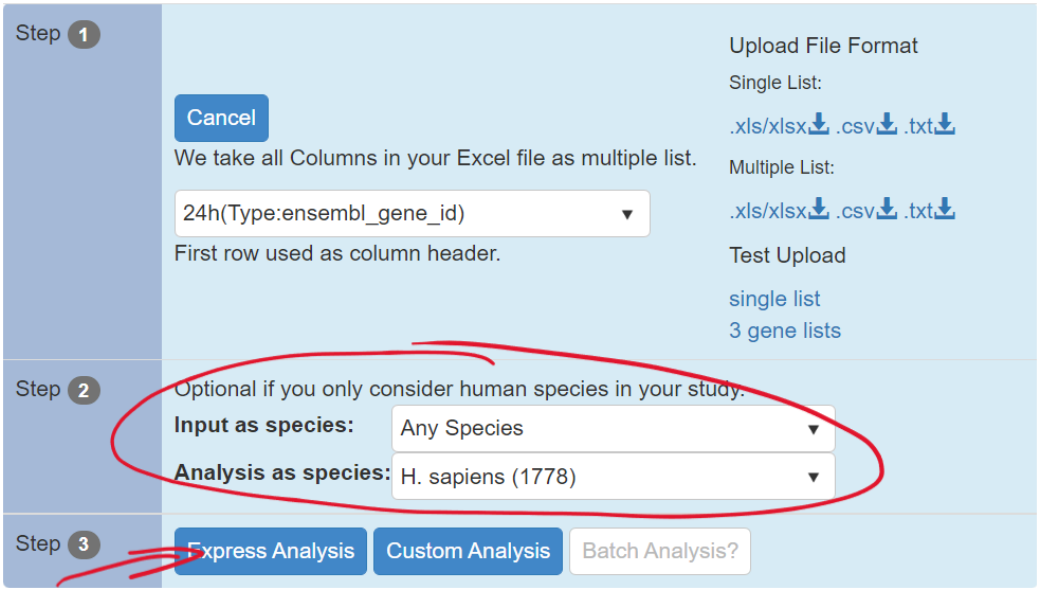
Metascape attempts to provide a number of "gene-list" type analyses under a single interface. In order to design Metascape, the programmers undertook a survey of existing tools and then attempted to make a single portal which offered all of the expected analyses. This makes Metascape a bit of a grab-bag, but it does offer some useful features - particularly multi-list meta-analysis.

By preparing a data file containing the differentially expressed genes from the two timepoint analyses as a .csv, we can run a Metascape analysis with both lists together. We can download this list from here: https://raw.githubusercontent.com/sjcockell/mmb8052/main/practicals/dat

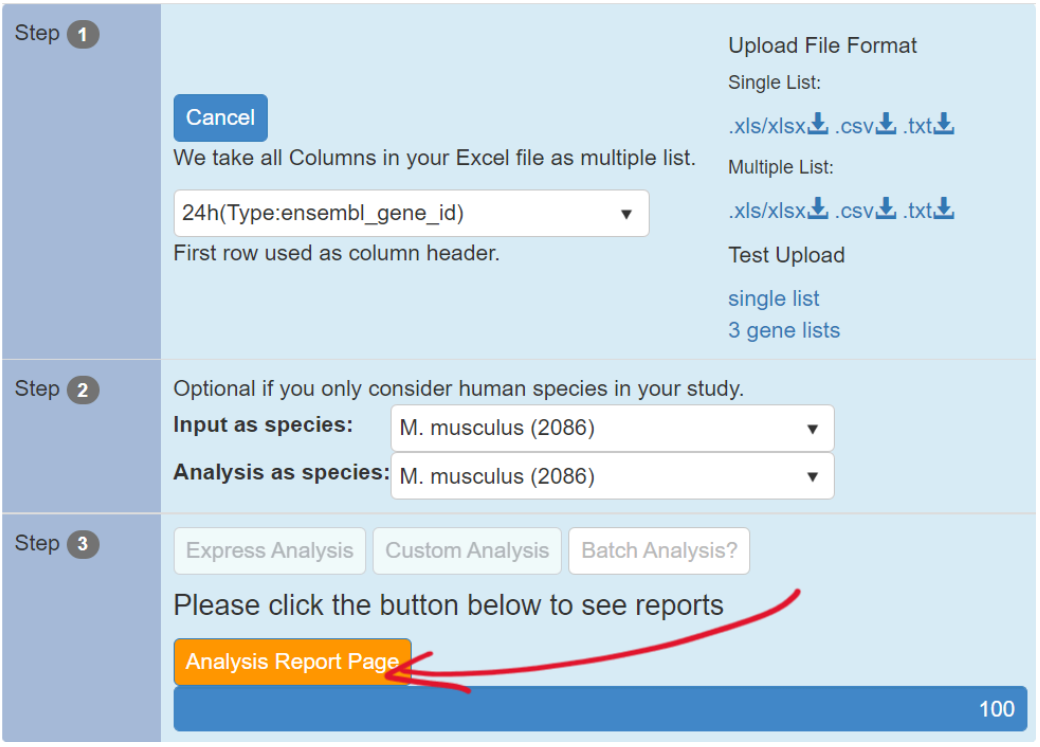
- Go to the Metascape website: https://metascape.org/gp/index.html#/main/step1
- Tick the "Multiple Gene Lists" tickbox
- Upload the file with the provided button



- Select "M. musculus" in both the drop-down species boxes
- Click "Express Analysis"



- The click through to the "Analysis Report Page" once the process is complete



CEMiTool

CEMiTool (Co-Expression Modules identification Tool) is a method that identifies co-expression gene modules from RNA-Seq data - that is, sets of genes which vary in the same way across an experiment, indicating some kind of regulatory relationship. As well as identifying modules, CEMiTool also performs analysis with those modules, providing visualisation and enrichment analysis at the module level.

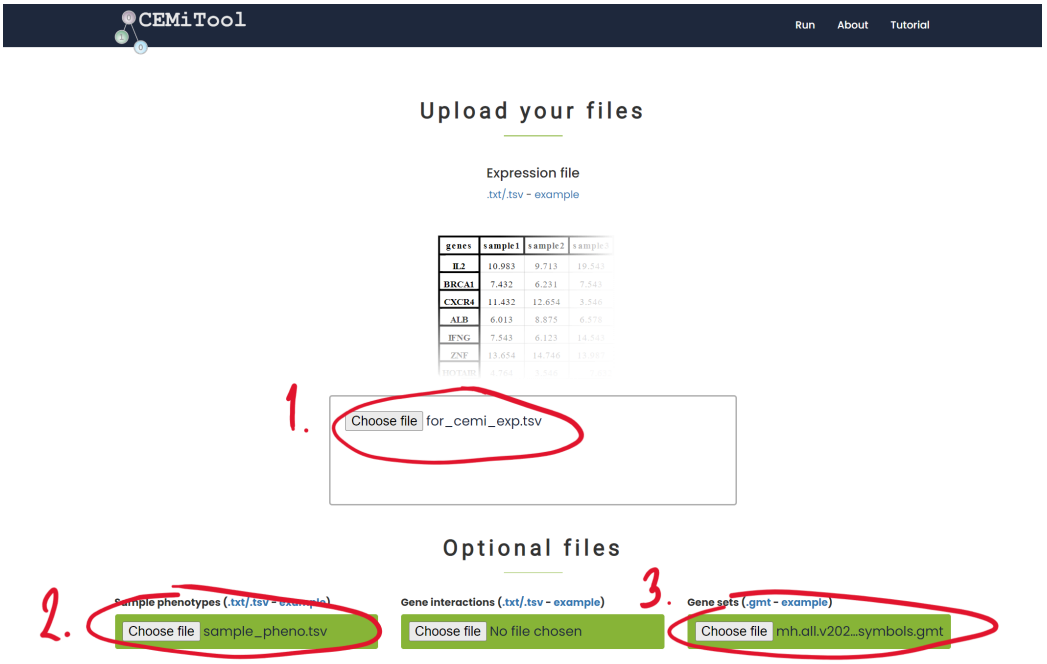
There is an R package for CEMiTool analysis, but we're taking a look at the web version here. To run this tool we need a few different files.

1. Our expression data, as tab-delimited text
2. Our sample phenotypes
3. A .gmt file of genesets we might be interested in

Each of these files can be downloaded from Github:

1. https://raw.githubusercontent.com/sjcockell/mmb8052/main/practica
2. https://raw.githubusercontent.com/sjcockell/mmb8052/main/practica
3. https://raw.githubusercontent.com/sjcockell/mmb8052/main/practica

In the CEMiTool web interface, load these files in the appropriate places:



Now we can click "Run CEMiTool", and wait for the results to be returned (this will take a few minutes).

Feel free to explore these tools as much as the remaining time in the session allows.

Summary

This final practical involves a lot of exploration of complex data, and so is left rather open-ended. Feel free to explore the resulting analysis as much as you like. It has to be said that the landscape for this kind of analysis is very rich for human experiments and for model vertebrates (e.g. mouse and rat), but the data available for similar analysis outside of these well-studied organisms can be problematic. Some classes of analysis should be possible regardless of the organism however, such as GO enrichment.

Over the nine practicals in this module, we have gone from learning the very basics of the Linux command line and R, to planning, organising and running quite complex analysis using primarily those tools. The principles of what we have learned can be applied to a wide range of different bioinformatics problems, especially as we have looked at how to navigate `man` and help information available in these tools.

Hopefully you now feel equipped to "have a go" with data you may encounter - especially during your research projects. If you do have any bioinformatics questions during the remainder of your MRes program (and beyond), do get in touch.